

Vysoké učení technické v Brně
Fakulta informačních technologií

Interpreter SOL26

Dokumentácia Implementácie

Obsah

1	Úvod	2
2	Popis návrhu Interpreteru	2
2.1	Sémantická analýza	2
2.2	Príprava prostredia na Interpretáciu	2
2.3	Vyhodnocovanie Programu	2
3	UML diagram	3
4	Dátové štruktúry a Triedy	4
4.1	Dátová Trieda Solclass	4
4.2	Dátová Trieda SolMethod	4
4.3	Dátová Trieda SolObject	4
4.4	Trieda Scope	4
4.5	Vstavane Triedy	4
5	Návrhové vzory OOP	4
5.1	Intepreter pattern	4
5.2	Singleton	4
6	Rozšírenia Interpretera	5
6.1	Modifikátory prístupu	5
6.2	Mechanizmus DoesNotUnderstand	5
7	Problémy pri riešení projektu	5
7.1	Ako spracovávať blok	5
7.2	Vyhodnocovanie self a super	5
8	Použitie AI nástrojov	5

1 Úvod

Tento dokument popisuje implementáciu Interpretera. Vysvetľuje ako je program rozdelený do častí pričom informuje ako je každá časť navrhnutá, aké objektovo orientované princípy používa a ako spolupracuje s inými časťami. Dokument je obohatený UML diagramom celého objektového návrhu.

V dokumente sa taktiež nachádzajú doplňujúce informácie o vývoji projektu (napr. používanie AI) a taktiež ako by sa teoreticky dal Interpreter rozšíriť.

2 Popis návrhu Interpreteru

Interpreter je logicky rozdelený na viacero častí pričom každá nasleduje až po dokončení predchádzajúcej.

2.1 Sémantická analýza

Ako prvá sa najprv uskutoční **kompletná** statická sémantická analýza, ktorá skontroluje vstupný program a pri odhalení chyby vracia korešpondujúci chybový kód. Tento princíp umožňuje odchytiť niektoré chyby už na začiatku spustenia Interpreteru pred vykonávaním samotného programu. Pri vyhotovovaní následných už statické chyby **nie sú** vôbec kontrolované.

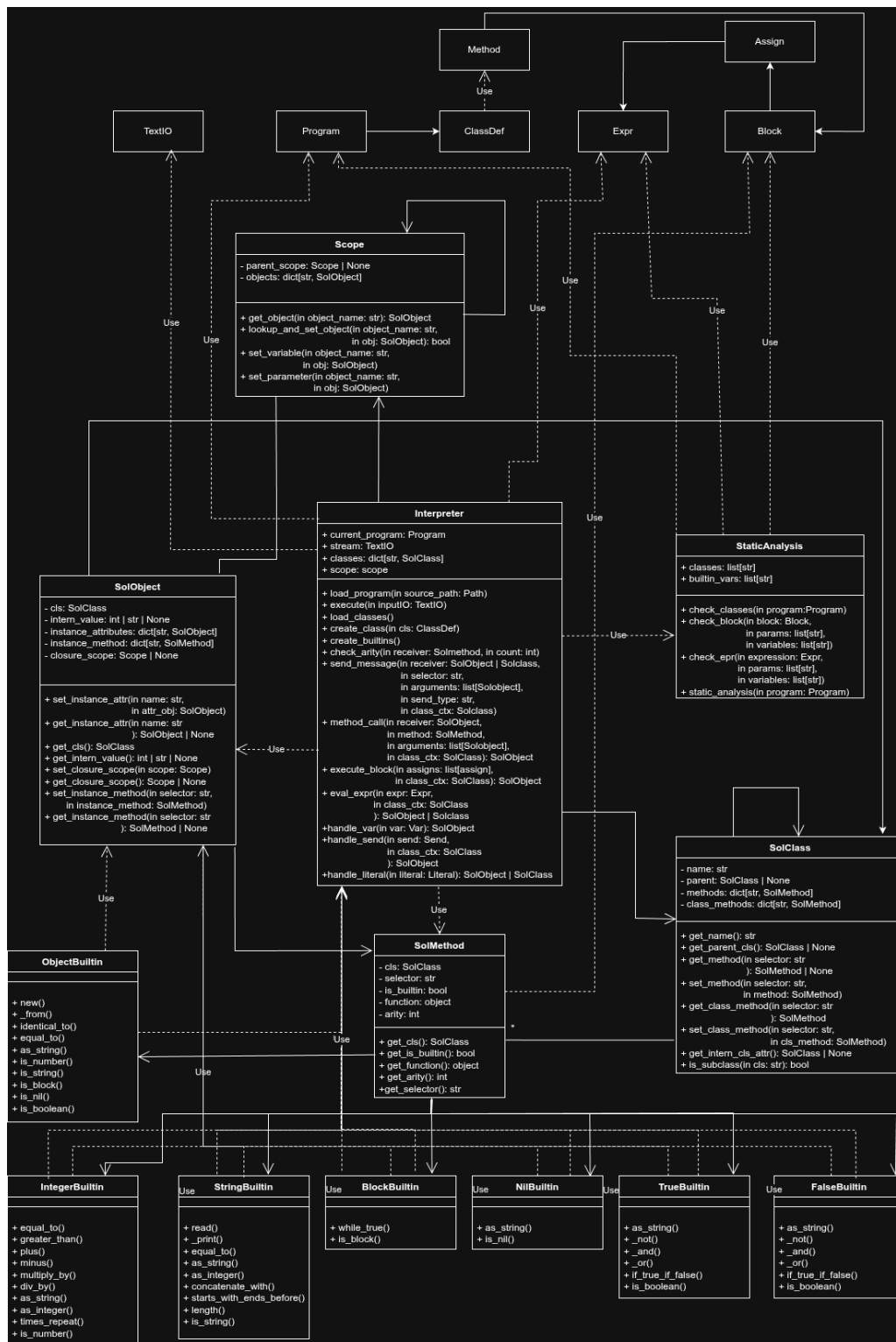
2.2 Príprava prostredia na Interpretáciu

Následne sa vytvoria objekty typu **SolClass** pre vstavané Triedy, a vytvoria sa ich metódy typu **SolMethod**, ktoré špecificky obsahujú odkaz na implementáciu metódy. Po vytvorení vstavaných tried sa rekurzívne vytvoria Triedy definované používateľom pričom ich metódy sú taktiež typu **SolMethod**, ktoré ale neobsahujú odkaz na funkciu implementácie ale vkladá sa do nich blok (postupnosť priradení) metódy. Ďalej sa vytvoria singleton objekty **true**, **false**, **nil** a globálne prostredie, do ktorého sú tieto objekty vložené. Nakoniec sa vytvorí objekt typu **Main**, ktorému je poslaná správa **run**.

2.3 Vyhodnocovanie Programu

Program sa vykonáva na princípe zasielania správ, kde správa obsahuje základné argumenty ako **príjemca**, **selektor** a **argumenty**, ale je obohatený aj typom správy a triednym kontextom. Typ správy závisí od toho ako bol vyhodnotený objekt príjemcu, to znamená, že či bol objekt zavolaný cez **self**, **super** alebo premennou. Triedny kontext je trieda, ktorá definuje danú metódu, v prípade bloku je to trieda, kde bol blok **definovaný**. Pri invokovaní metódy skrytej za selektorom sa rozlišuje či je daná metóda vstavaná, čo spôsobí invokáciu definovanej metódy, alebo je metóda definovaná užívateľom, čo spôsobí invokáciu bloku uloženého v metóde. Pri invokácii bloku program postupne prechádza jednotlivé priradenia a spúšťa ich následné vyhodnocovanie. Jednotlivé výrazy sú následne vyhodnocované na základe ich typu pričom Interpreter spúšťa ich korešpondné vyhodnocovacie funkcie.

3 UML diagram



Obr. 1: Diagram Tried Interpretera

4 Dátové štruktúry a Triedy

4.1 Dátová Trieda Solclass

Trieda SolClass ukladá jednotlivé triedy. Každá trieda má svoje meno, odkaz na nadtriedu typu Solclass, slovník svojich metód, a pri vstavovaných triedach slovník triednych metód.

4.2 Dátová Trieda SolMethod

Trieda SolMethod ukladá jednotlivé metódy, pri každej metóde ukladá odkaz na triedu, ktorej patrí, svoj selektor, pravdivostnú hodnotu či je vstavaná, svoju aritu. Atribút ktorý obsahuje metódu môže byť odkaz na vstavanú metódu, ktorú implementuje Interpreter alebo obsahuje samotný blok s postupnosťou priradení, ktoré sa majú previesť.

4.3 Dátová Trieda SolObject

Táto trieda Vytvára objekty SOL26, kde každý objekt má odkaz na svoju triedu, **slovník** obsahujúci objekty inštančných atribútov, v prípade triedy String a Integer taktiež interný inštančný atribút. Ak je objekt vytvorený z triedy Block, tak taktiež obsahuje odkaz na objekt triedy **Scope** a metódu **value**, ktorej dokáže zaslať správu.

4.4 Trieda Scope

Trieda Scope popisuje prostredie v ktorom sú uložené jednotlivé objekty, ktoré sú skryté za názvom premennej v slovníku. Každé prostredie obsahuje odkaz na predchádzajúce(nadriadené) prostredie, čo umožňuje jednoduché vyhľadávanie v nadriadených prostrediach. Pri spustení Interpretera sa do Globálneho prostredia, ktoré nemá žiadne nadriadené prostredie vytvoria singleton objekty **true**, **false** a **nil**, ktoré sa vždy pri použití v kóde vždy hľadajú cez jednotlivé prostredia rekurzívne.

4.5 Vstavované Triedy

Triedy **ObjectBuiltin**, **IntegerBuiltin**, **StringBuiltin**, **NilBuiltin**, **BlockBuiltin**, **TrueBuiltin** a **FalseBuiltin** obsahujú jednotlivé implementácie vstavovaných metód. Tieto triedy neobsahujú ako sa majú jednotlivé objekty tried chovať, ani nepopisujú priamo triedy jazyka, slúžia čiste na implementáciu metód podľa zadania.

5 Návrhové vzory OOP

5.1 Interpreter pattern

Interpreter pattern je v tomto kóde popísaný hlavne v triede **Interpreter**, v ktorej sú popísané metódy na spracovanie každého typu uzlu. Každý typ uzlu v AST je vyhodnocovaný svojou metódou. Napríklad pri spracovaní výrazu, existuje viacero rôznych typov výrazov, kde každý má rôzny význam, sa volajú ich korešpondné metódy. Pri tomto vzore je veľmi jednoduché pridanie nového typu výrazu, stačí pridať iba novú podmienku pre daný typ výrazu a implementovať jeho metódu.

5.2 Singleton

Singleton pattern je používaný pri objektoch tried **true**, **false** a **nil**, ktoré sa vytvoria pre samotným interpretovaním programu. Pri prechádzaní programového stromu a nájdení hodnoty uzlu **true**, **false** alebo **nil** sa hľadá v prostredí daný objekt a nikdy sa v kóde nevytvára nový. Tento vzor umožňuje porovnávanie objektov pomocou **identical_to**.

6 Rozšírenia Interpretera

6.1 Modifikátory prístupu

Modifikátory prístupu by mohli byť implementované pridaním nového atribútu do triedy `SolMethod`, ktorý by určoval typ metódy (`public`, `private`, `protected`). Pri zaslaní správy by sa kontrolovalo aký má metóda typ, v prípade, že je chránená tak by sa kontrolovalo či je príjemca `self` alebo `super`. Pri sémantickej analýze by sa kontrolovalo, že zdedená metóda nemôže byť menej prístupná ako metóda nadtriedy.

6.2 Mechanizmus `DoesNotUnderstand`

Najskôr by sa musela implementovať vstavaná trieda `Message`, ktorá dedí z `Object`. Trieda by obsahovala atribúty `selector(String)` a `argumenty(list stringov alebo objektov)` a počet argumentov (`Integer`), ktoré by sa pridali do `SolObject`. Následne by sa implementovala vstavaná metóda `DoesNotUnderstand` v triede `ObjectBuiltin`, ktorá by vyvolala chybu 51. Nakoniec by sa upravila metóda `send_message` v Triede `Interpreter`, ktorá by pri nájdení chyby vytvorila Objekt typu (`Message`) so všetkými atribútmi chybovej správy a potom zaslala by správu `DoesNotUnderstand:` Objektu príjemcu s argumentom `Message` objektu. Pri zaslaní správy `"DoesNotUnderstand:"` by sa zároveň kontrolovalo či sa znovu nezavolá `"DoesNotUnderstand:"` napríklad pomocnou vlajkou.

7 Problémy pri riešení projektu

7.1 Ako spracovávať blok

Spracovanie objektu bolo zložité, prvý návrh implementácie bol správať sa ku nemu ako ku metóde ale to bol zlý nápad z dôvodu inštančných atribútov a definovania jeho prostredia. Nakoniec sa to vyriešilo tak, že `SolObjektu` boli pridané atribúty, ktoré uchovávajú blokové prostredie a uchovávajú `SolMethod`, ktorá obsahuje samotnú definíciu bloku.

7.2 Vyhodnocovanie `self` a `super`

Problém spočíval v určení vyhodnocovania `self` a `super` a hľadani metódy pri použití `self` a `super`. Vyriešenie problému nastalo až po pridaní sekcie 1.2.12, kde som pochopil ako to celé má fungovať a veľká časť implementácie zasielania správ je inšpirovaná práve týmto dodatkom. Hľadanie metódy pri použití `self` a `super` som sa vyriešilo pridaním odkazu na triedu pri metóde a pridaním typu zaslanej správy triedneho kontextu pri zasielaní správy.

8 Použitie AI nástrojov

V tomto projekte boli použité AI nástroje **ClaudeAI - Anthropic** a **ChatGPT**.

AI nástroje boli použité na:

- Vysvetlenie častí projektu (napríklad AKO majú fungovať jednotlivé vstavané metódy)
- Vysvetlenie princípov kontejnerizácie
- Vysvetlenie princípov syntaxe a funkcií jazyka Python a PHP
- Debugovanie nefunkčných častí projektu, prípadne hľadanie chýb v kóde
- Vytváranie testov

- Vysvetľovanie princípov OOP
- Pri dokumentácii čisto na hľadanie gramatických chýb